

# **RUM** **Community Group**

2026-04-08 Edition

# Agenda

- Performance Timeline option per-paint (performance-timeline[#228](#)) (Michal)
- TAO and ServerTiming Header Adoption Updates (Cliff)
- RUM ABCs (AI, Bots and Crawlers) (Philip et. al)

# Logistics

- Meeting cadence
  - 2nd Wednesday of the Month
  - 60 minutes
  - 10am ET
- Agenda Document
  - ([bit.ly/rumcg-agenda](https://bit.ly/rumcg-agenda))
- Google Meet for Meetings
  - sub to [rumcg-participants](#) for invites
- Chat on Web Performance Slack  
#w3c-rum-community-group
  - ([invites](#))
- [public-rumcg@w3.org](mailto:public-rumcg@w3.org)
  - (for discussions)
- RUM CG Github Repo
  - [github.com/w3c-cg/rum](https://github.com/w3c-cg/rum)
  - Group details
  - Links
  - Meeting minutes
- RUM CG Tracking Project
  - [github.com/orgs/w3c-cg/projects/1](https://github.com/orgs/w3c-cg/projects/1)
- Meeting Minutes via [fireflies.ai](https://fireflies.ai)
  - AI summary + transcript will be posted after
  - Full video recording also available

# Next Meeting

- Wed **May 13th** @ 10am EST
  - 2nd **Wednesday** of each month
- Topics?
  - ?
- Submit & Vote topics @ [bit.ly/rumcg-agenda](https://bit.ly/rumcg-agenda)

## Topics Backlog

- ~~Better tracking/visibility into vendor positions (March 14)~~
- ~~Top tracking issues (e.g. this [lighthouse issue](#)) (March 14)~~
- ~~Baseline Project~~
- ~~Funding browser work for features~~
- ~~Header adoption (TAO, Server timing)~~
- ~~Browser vendor presentations~~
- Outreach to other groups (i.e. 3rd Parties, CDNs)
- Ad Blockers & tracking protection
- Implementation best practices
  - When to fire the beacon/how to deliver
  - SPA support
- Making sure we don't step on each other's toes (i.e. clearing RT buffers, multiple vendors on a page, etc.)
- Open source
- Securing the beacon
- ~~RUM Archive~~
- ~~Callstacks from input delays (Issac Gerges [thread](#) in #general)~~
- ~~Container Timing awareness~~
- ServerTiming / trailers
- Bot and bad actor detection (bad CWW bots)
- AI browsers

**let's do this**

# Performance Timeline option per-paint

- <https://github.com/w3c/performance-timeline/issues/228>



mmocny opened 2 weeks ago

Member ...

There are a bunch of "paint timings" that are logically scheduled and presented together:

- Paint Timing, LCP, and Element/Container Timing
- Long Animation Frames
- Event Timing
- Layout Shifts
- (Soft Navigations and ICP)

Especially with Long Animation Frames, it can be very useful to cluster related entries "by paint". Usually, this happens automatically because a PerformanceObserver is invoked typically once per paint in an idle period.

But there are a myriad cases where many animation frames' worth of entries are buffered and reported together:

- The initial observe() callback with buffered: true
- Any time the main thread is busy we might defer invoking the observer callback...
- Sometimes certain performance entries themselves are internally buffered and then flushed to perf timeline in chunks (i.e. Event Timing)

In such cases, it is left up to the developer to do their own slicing and grouping based on e.g. startTime and paintTime values.

This becomes exceedingly more difficult when you have multiple separate PerformanceObserver registered for different entry types.

Proposal: Add a PerformanceObserver option to opt-in to "per-paint" callback invocation. Whenever there are multiple entries that belong to a different animation frame, they will be invoked in a separate callback. That way, a simple `for (let entry of list.getEntries())` loop will be for a single "unit of work".

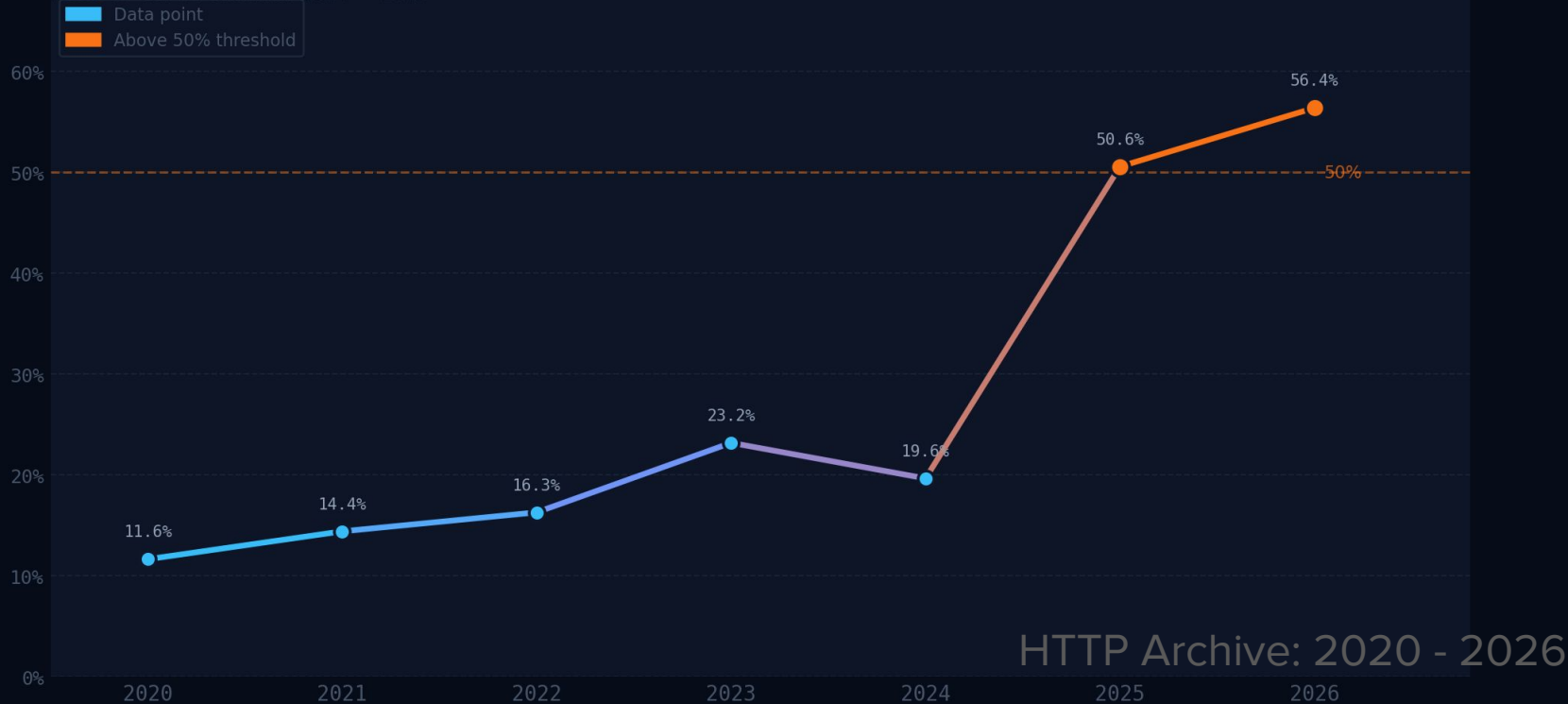
Additionally, if there are multiple observers registered, we can invoke the callbacks in an interlaced fashion.

# Server-Timing Header Adoption

ADOPTION METRICS / 2020-2026

## Adoption Rate

Notable surge between 2024 → 2025



# Server-timing Adoption Rate by CrUX Rank

| Rank       | Pages w/ Timing | Total Pages | Adoption % |
|------------|-----------------|-------------|------------|
| 1,000      | 845             | 1.2K        | 70.4%      |
| 5,000      | 3.6K            | 5.0K        | 71.5%      |
| 10,000     | 4.4K            | 6.2K        | 71.4%      |
| 50,000     | 3.4K            | 14.9K       | 69.1%      |
| 100,000    | 4.0K            | 56.1K       | 65.7%      |
| 500,000    | 3.2K            | 905.1K      | 62.5%      |
| 1,000,000  | 4.1K            | 636.8K      | 60.5%      |
| 5,000,000  | 3.2K            | 2.18M       | 57.6%      |
| 10,000,000 | 3.5K            | 7.00M       | 56.1%      |
| 50,000,000 | 5.8K            | 61.81M      | 55.8%      |

ADOPTION RATE SCALE



HTTP Archive: Feb 2026



# RUM ABCs (AI, Bots and Crawlers)

- Most AI bots don't execute JavaScript or send an mPulse beacon
- Our security team has helped us identify which of our beacons did come from AI bots (~0.005%)
- The most interesting thing is the difference in which pages are viewed

# Four types of AI Bots

- AI Training Crawlers

Used to train AI models

- AI Search Crawlers

Used to provide search summaries

- AI Fetchers

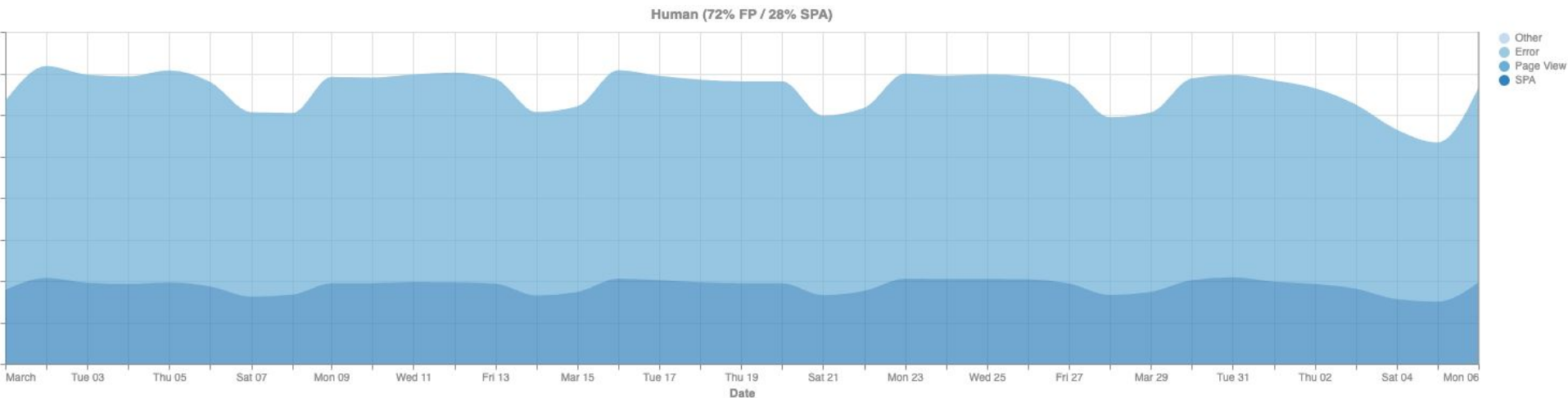
Act on a user's behalf to answer a very specific question via simple web requests. These do not maintain state.

- AI Agents

Act on a user's behalf to perform complex tasks like "but me a good pair of sneakers at the best rate". These maintain state and have memory.

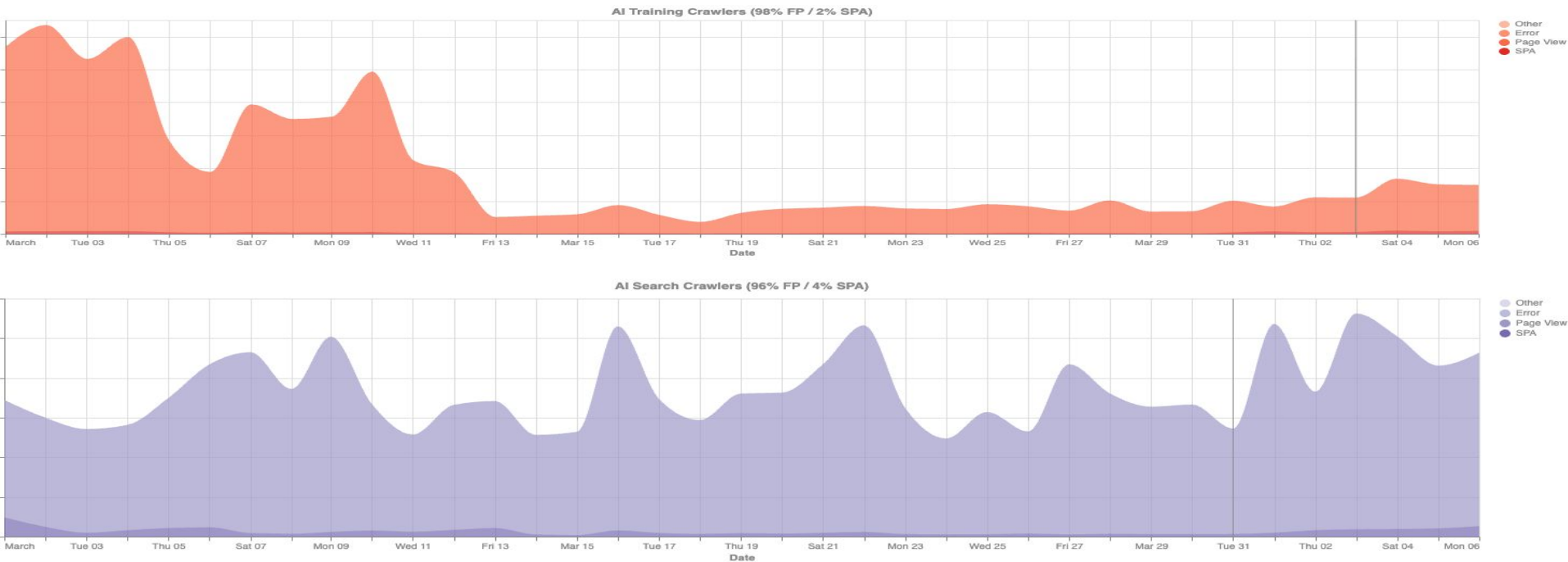
# Human v/s Bot Behavior(u)r

Human traffic has a 70-30 split between full page (including bfcache) and SPA hits.



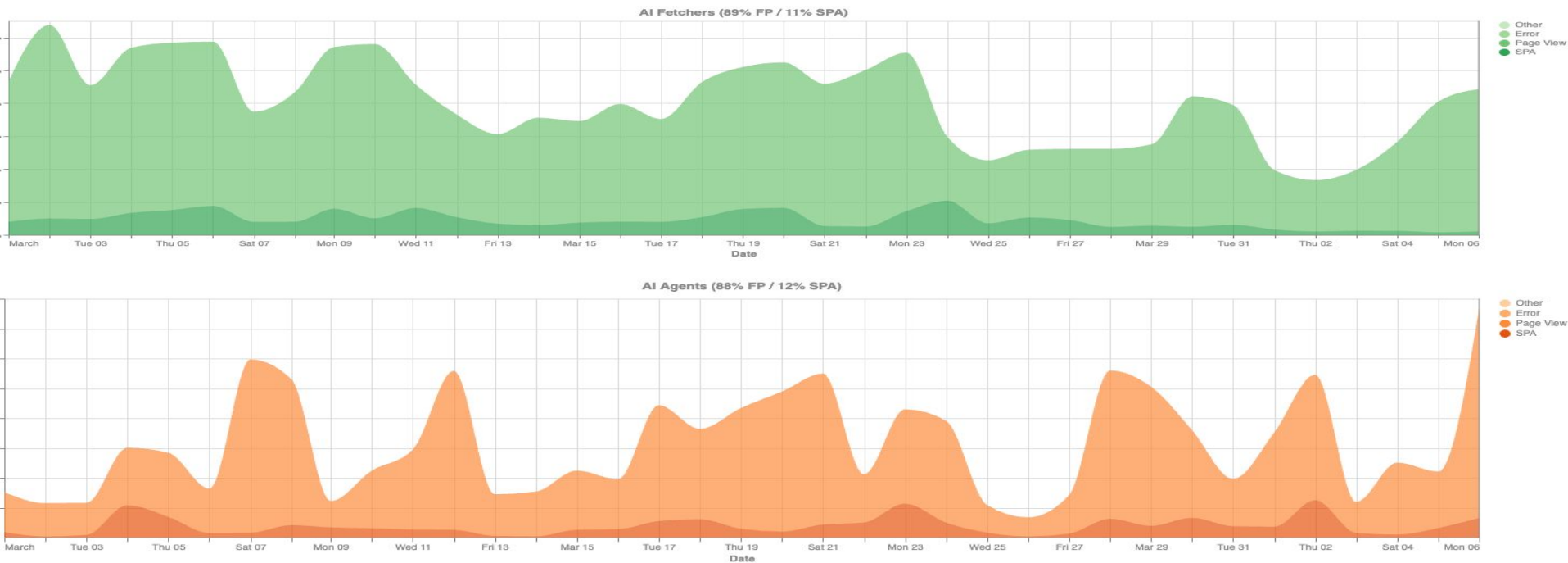
# Human v/s Bot Behavior(u)r

Training & Search **Crawlers** have a 95-5 split between full page and SPAs



# Human v/s Bot Behavior(u)r

Fetchers and Agents have an 88-12 split between full page and SPAs



# Human v/s Bot Behavior(u)r

## Summary

- Human: 72% Full Page v/s 28% SPA
- Training Crawlers: 98% Full Page v/s 2% SPA
- Search Crawlers: 96% Full Page v/s 4% SPA
- Fetchers: 89% Full Page v/s 11% SPA
- Agents: 88% Full Page v/s 12% SPA

Our data also shows 80% bounce rate for AI bots, but I'm not confident in that.

# RUM ABCs (AI, Bots and Crawlers)

## Questions:

- How much do we think bots present in RUM data?
- How could / should AI crawlers identify themselves?
  - User Agent, IP, ASN?
  - Web Bot Auth (<https://github.com/cloudflare/web-bot-auth>)
  - <https://github.com/ai-robots-txt/ai.robots.txt>
- How could / should AI/agentic browsers identify themselves?
  - Comet, Atlas, Neon, etc
  - Do "they" even care about performance?